

Deep Learning (CAI-461)

Objectives:

- EfficientNet

EfficientNet:

EfficientNet is a family of convolutional neural networks (CNNs) designed for image classification and other visual recognition tasks. It was introduced by researchers at Google AI in 2019 as a solution to improve the accuracy and efficiency of deep learning models. The key motivation behind EfficientNet is to achieve a better trade-off between accuracy and computational efficiency, making it suitable for a wide range of devices and applications, from mobile devices to large-scale cloud computing systems.

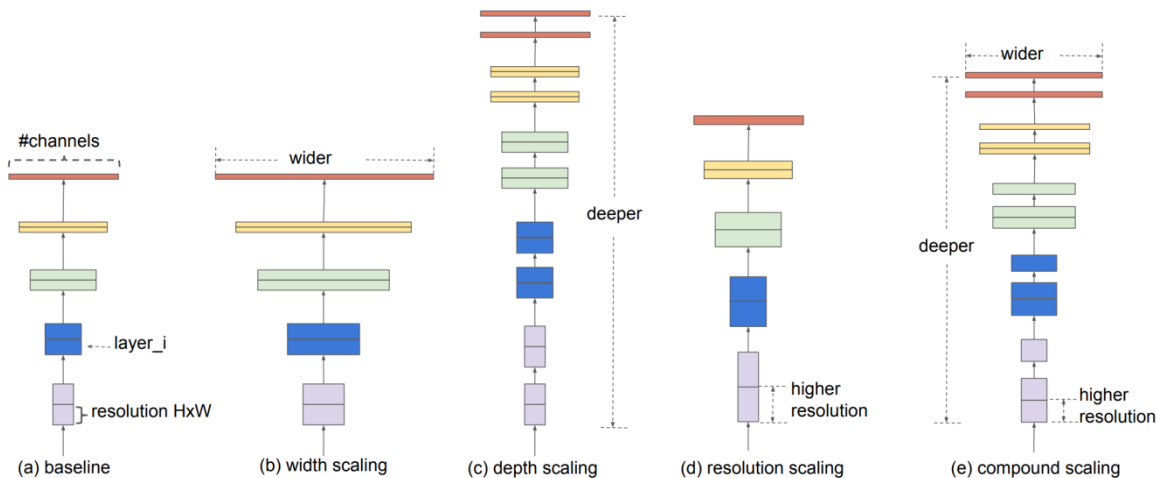


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

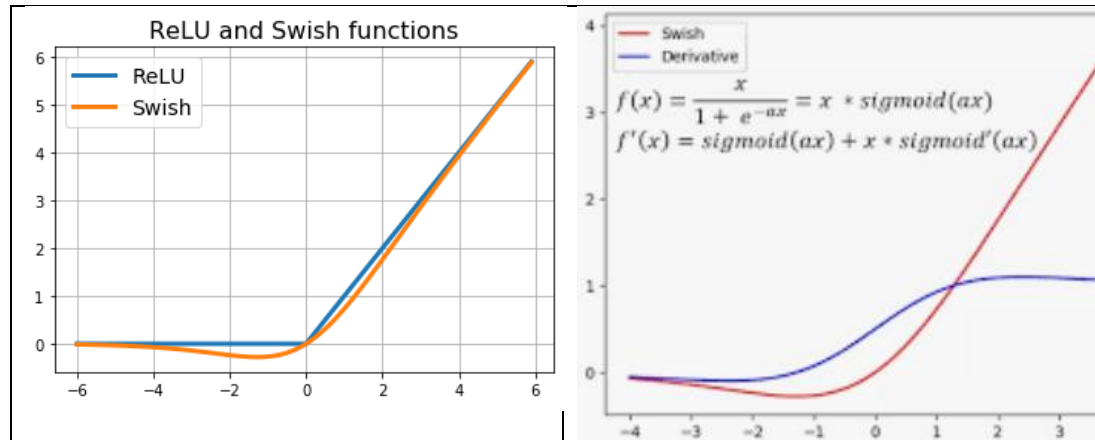
1. Compound Scaling:

- The main idea behind EfficientNet is to scale up the model efficiently using a technique called **compound scaling**. Instead of simply increasing the depth (number of layers) of a model, EfficientNet scales three dimensions:
 - **Depth:** The number of layers in the network.
 - **Width:** The number of channels in each layer.
 - **Resolution:** The size of the input image.
- Traditional scaling methods tend to focus on just one of these dimensions (e.g., just increasing depth), but EfficientNet scales all three dimensions in a balanced way, which leads to better overall performance without exponentially increasing computational cost.

2. Baseline Model (EfficientNet-B0):

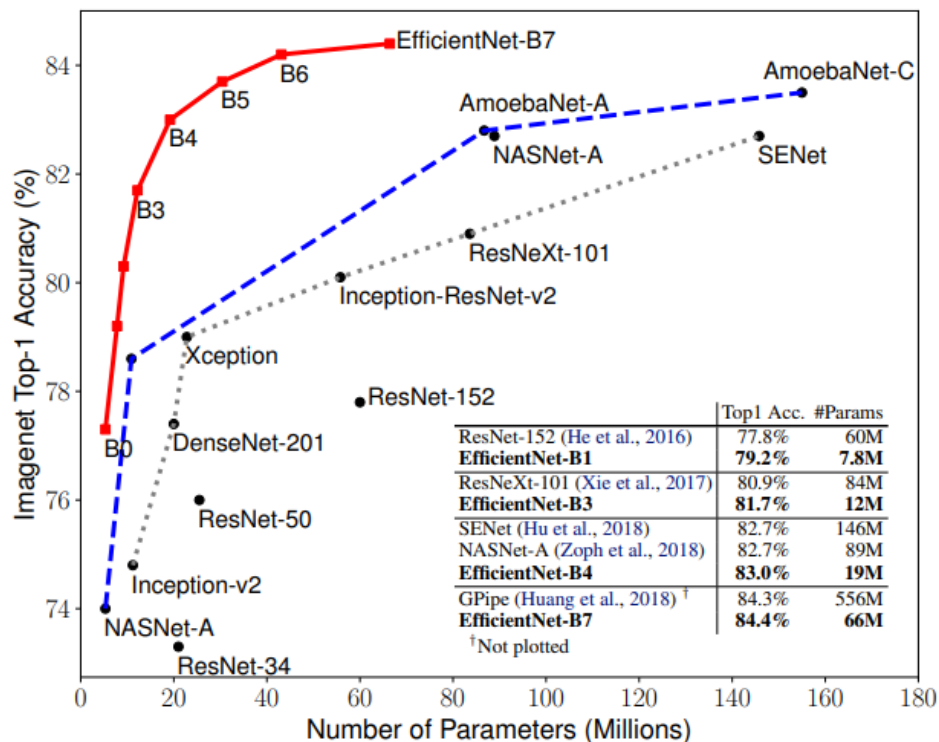
- EfficientNet starts with a baseline model, EfficientNet-B0, which is designed to be both highly efficient and effective. This model uses **Mobile Inverted Bottleneck Convolutions (MBConv)** and **Swish activation functions** for better performance and computational efficiency.
 - MBConv is a variant of depthwise separable convolutions, which are computationally cheaper than traditional convolutions.
3. **Swish Activation Function:**
- EfficientNet uses the **Swish activation function**, a self-gated activation function, as an alternative to the commonly used ReLU. Swish has been shown to outperform ReLU in many deep learning tasks.
4. **Search for Optimal Architecture:**
- To design EfficientNet-B0, the authors performed a neural architecture search (NAS) to find the best possible architecture for the base model. NAS is an automated process where the architecture of the network is optimized to achieve the best accuracy with minimal computational cost.
5. **Scalability and Variants:**
- Once EfficientNet-B0 was established, the researchers applied compound scaling to create a family of models (B0 through B7). Each model in the EfficientNet family scales in terms of depth, width, and resolution, with larger models like EfficientNet-B7 being more accurate but also more computationally expensive.
 - The efficient scaling allows EfficientNet to achieve state-of-the-art accuracy while being much more computationally efficient than traditional models like ResNet or DenseNet.
- **MBConv Blocks** (Mobile Inverted Bottleneck Convolutions):
 - EfficientNet uses a novel type of convolutional block called **MBConv**, which is a variant of depthwise separable convolutions.
 - **Depthwise separable convolutions** are more computationally efficient than traditional convolutions, as they separate the convolution operation into two parts: a depthwise convolution (filtering individual channels) and a pointwise convolution (combining them).
 - **Inverted Bottleneck design:** The input to each block has fewer channels, then expands into a larger number of channels, and then contracts back to a smaller number of output channels. This design improves the network's capacity without significantly increasing computational complexity.
 - **Squeeze-and-Excitation:**
 - In the EfficientNet-B0 architecture, each block has a **Squeeze-and-Excitation (SE)** module. The SE module is used to recalibrate channel-wise feature responses by learning dynamic channel-wise importance. This helps the model learn which features are most important.
 - **Swish Activation Function:**

- EfficientNet uses the **Swish** activation function instead of the more commonly used ReLU. Swish has been shown to outperform ReLU in many deep learning applications, as it allows for smoother gradients and less information loss in deeper networks.



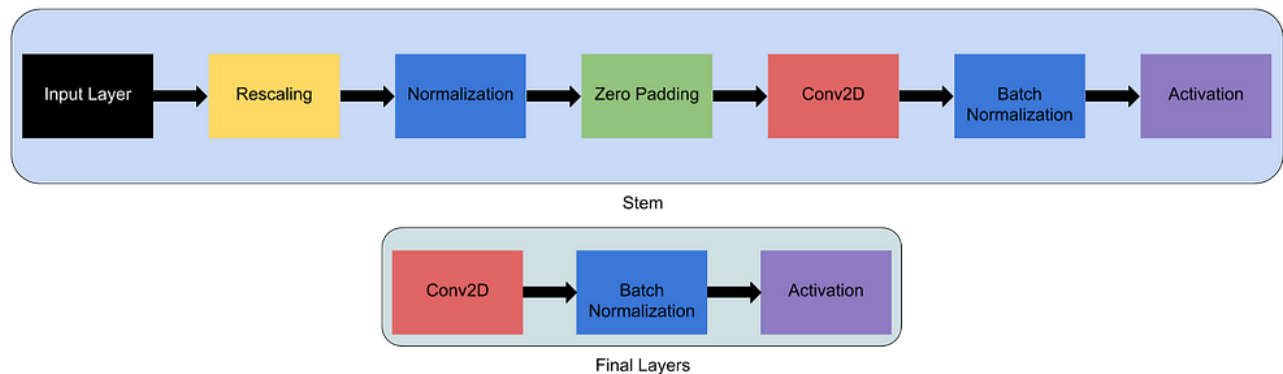
- Efficient Layer Arrangement:**

- The EfficientNet architecture uses a sequence of **convolutional layers** followed by **batch normalization** and **activation functions** (specifically Swish) in a structured manner.
- The model also uses **global average pooling** followed by a softmax layer for classification tasks.



Common Things In All

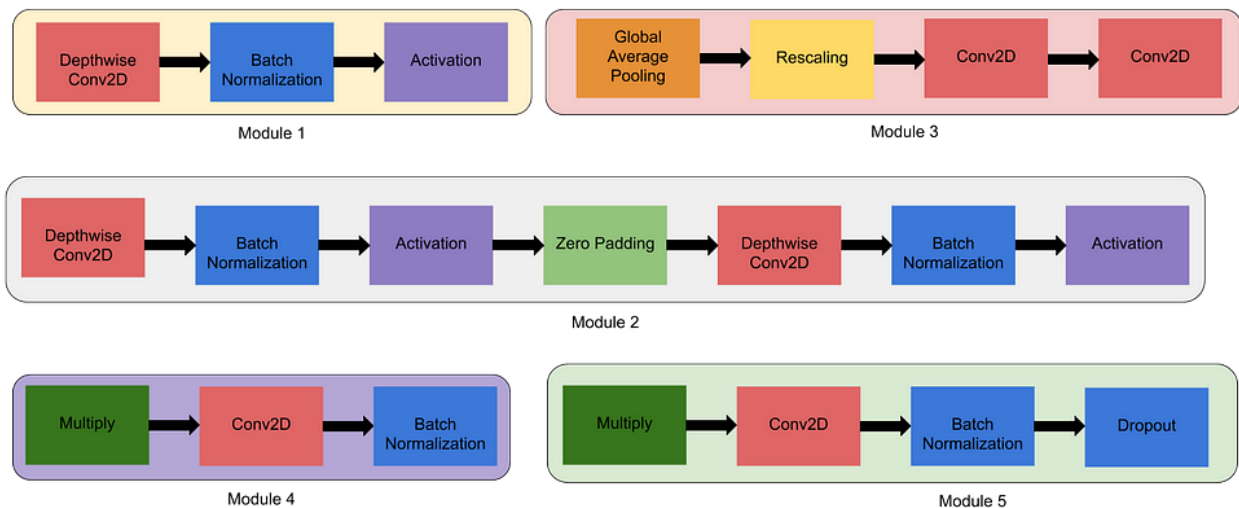
The first thing in any network is its stem after which all the experimenting with the architecture starts which is common in all the eight models and the final layers.



After this each of them contains 7 blocks. These blocks further have a varying number of sub-blocks whose number is increased as we move from EfficientNetB0 to EfficientNetB7.

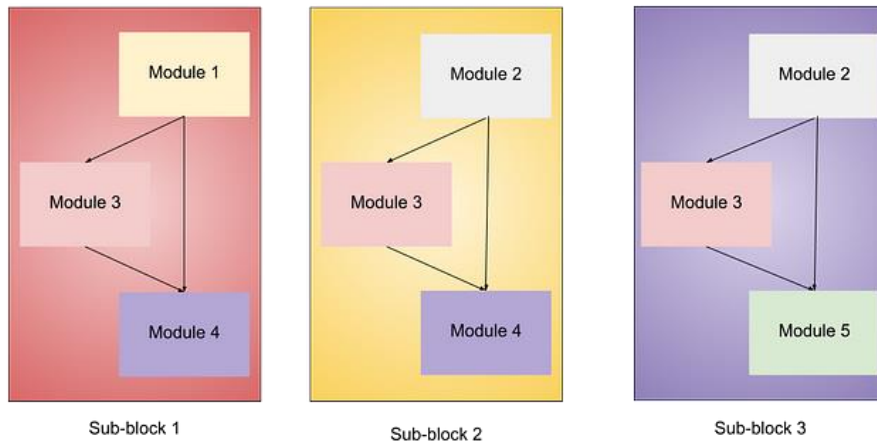
If you count the total number of layers in EfficientNet-B0 the total is 237 and in EfficientNet-B7 the total comes out to 813!! But don't worry all these layers can be made from 5 modules shown below and the stem above.

5 modules we will use to make the architecture.



- **Module 1** — This is used as a starting point for the sub-blocks.
- **Module 2** — This is used as a starting point for the first sub-block of all the 7 main blocks except the 1st one.
- **Module 3** — This is connected as a skip connection to all the sub-blocks.
- **Module 4** — This is used for combining the skip connection in the first sub-blocks.
- **Module 5** — Each sub-block is connected to its previous sub-block in a skip connection and they are combined using this module.

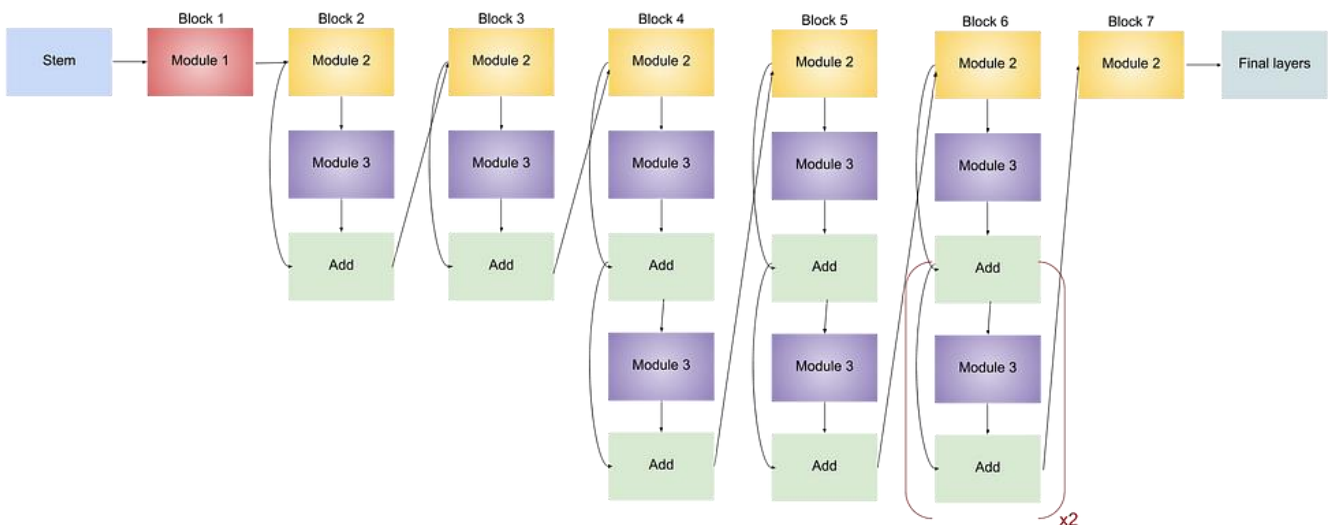
These modules are further combined to form sub-blocks which will be used in a certain way in the blocks.



- **Sub-block 1** — This is used only used as the first sub-block in the first block.
- **Sub-block 2** — This is used as the first sub-block in all the other blocks.
- **Sub-block 3** — This is used for any sub-block except the first one in all the blocks.

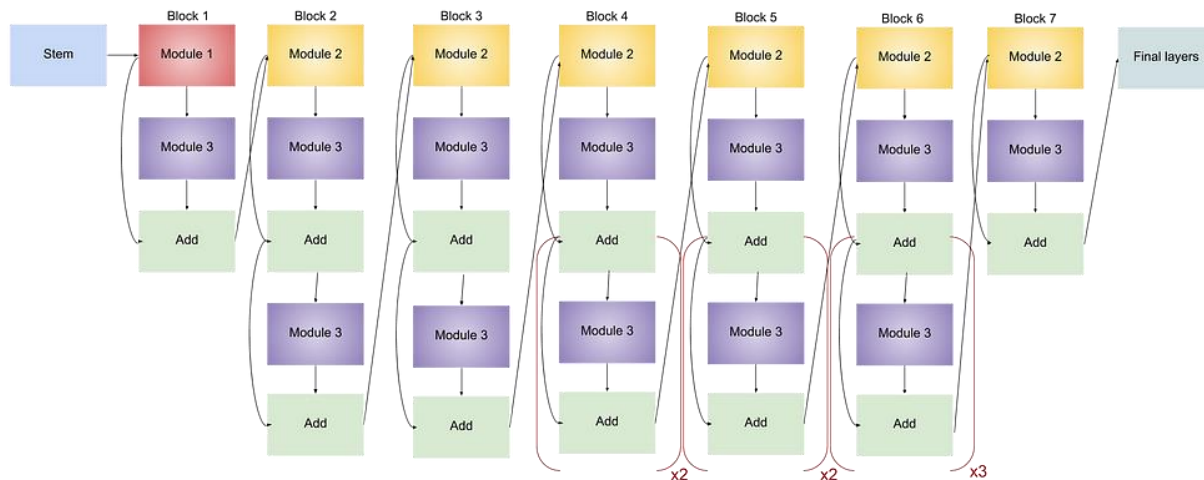
Till now we have specified everything that will be combined to create the EfficientNet models so let's get started.

EfficientNet-B0



Architecture for EfficientNet-B0. (x2 means that modules inside the bracket are repeated twice)

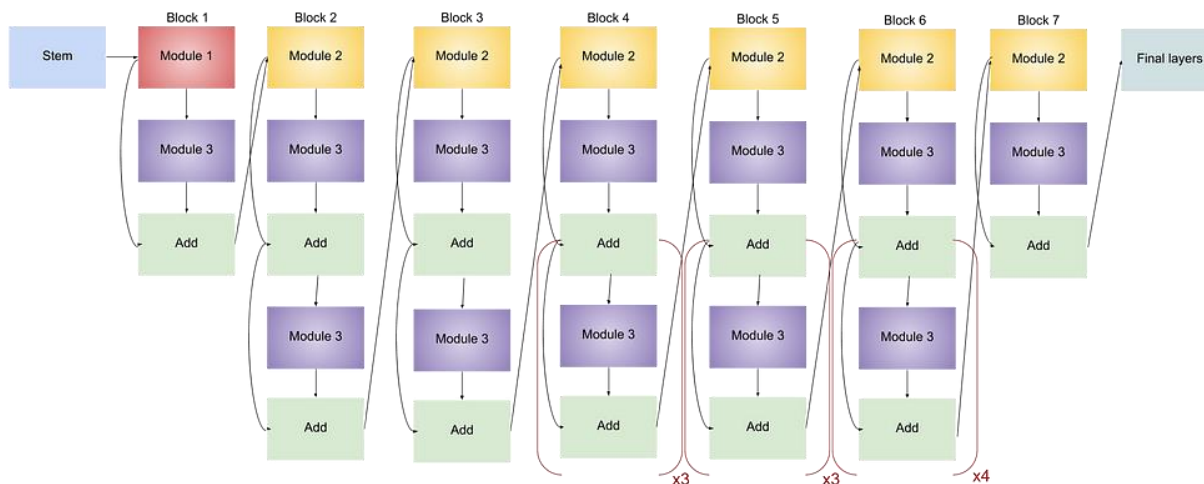
EfficientNet-B1



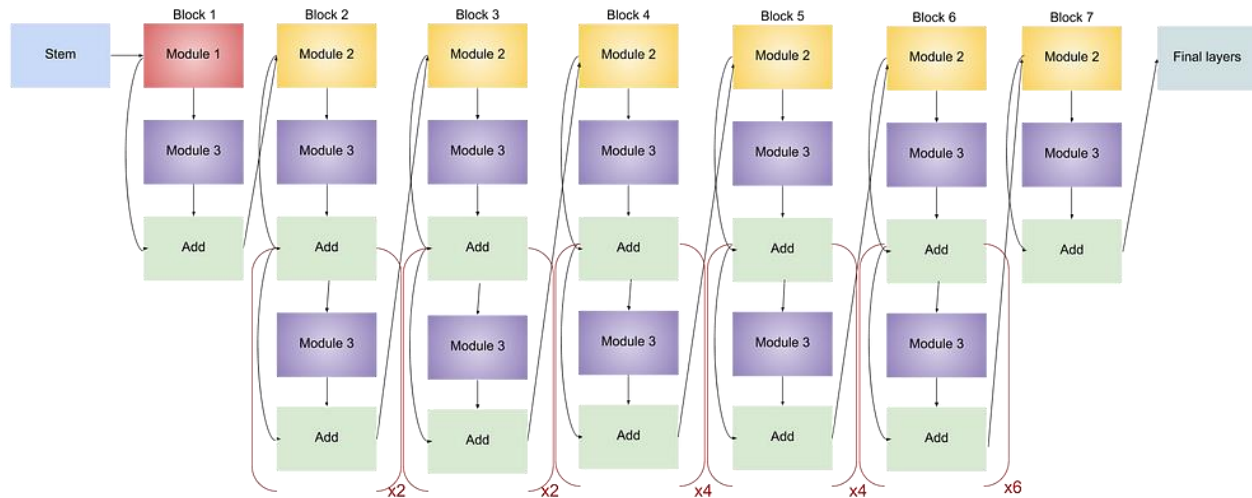
EfficientNet-B2

Its architecture is the same as the above model the only difference between them is that the number of feature maps (channels) is varied that increases the number of parameters.

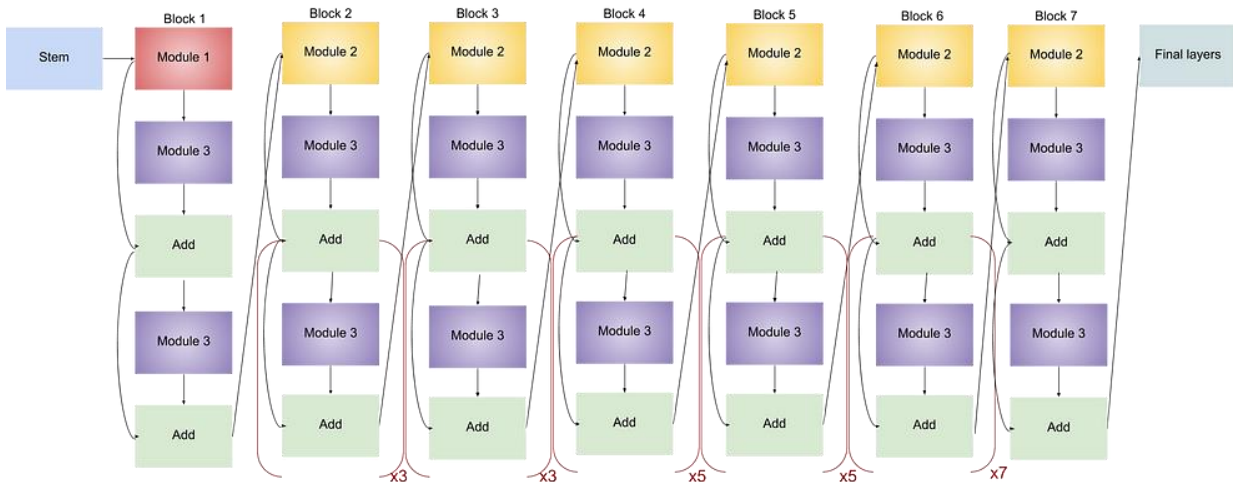
EfficientNet-B3



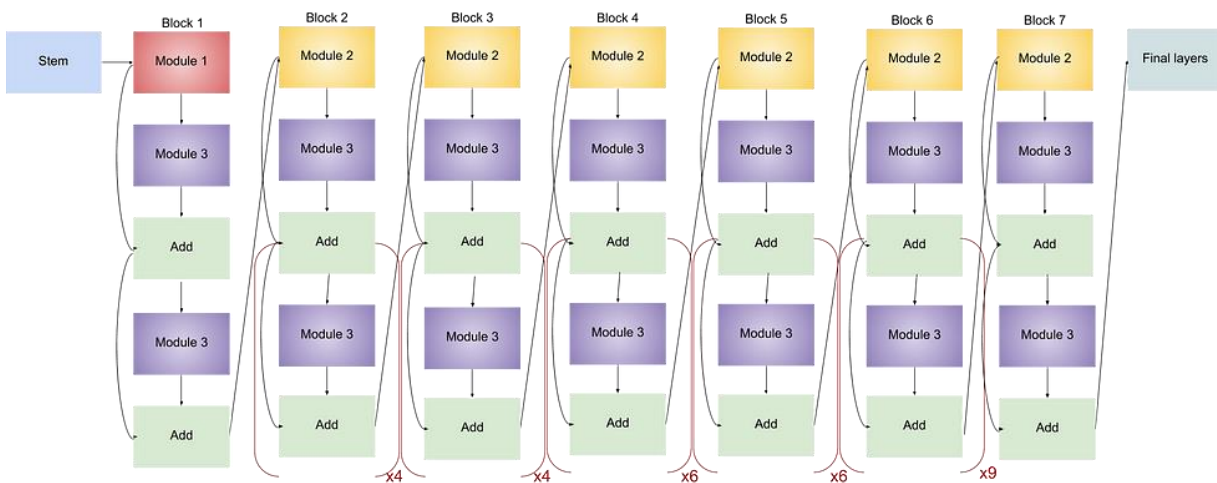
EfficientNet-B4



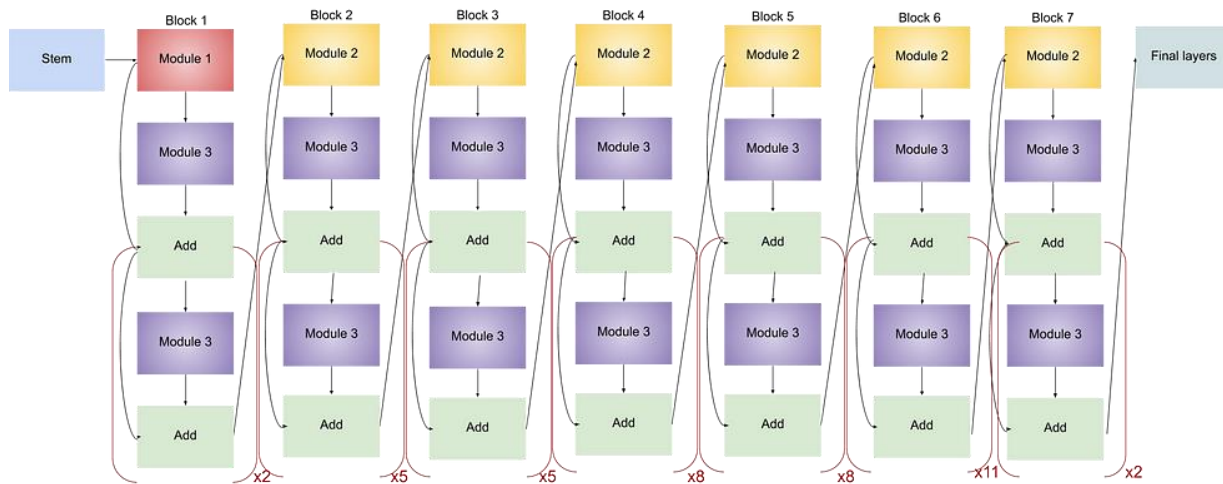
EfficientNet-B5



EfficientNet-B6



EfficientNet-B7



It's easy to see the difference among all the models and they gradually increased the number of sub-blocks.